IRBench: Instruction Robustness Benchmark for Large Language Models

P3 Team
Missouri University Science & Technology

April 6, 2026

Abstract

1 Introduction

2 Literature Review

2.1 Instruction-Following Benchmarks

Instruction-following evaluation has progressed from broad capability assessment to increasingly fine-grained measurement of constraint adherence. InFoBench introduced one of the clearest decompositional perspectives by proposing the Decomposed Requirements Following Ratio (DRFR), which breaks a complex instruction into simpler requirements and evaluates compliance at that finer level rather than only at the response level (Qin et al. [2024]). This line of work is important because it shows that instruction following is not a monolithic capability: a model may satisfy some requirements while failing others. TOWER extended this idea by arguing that not all parts of a complex instruction are equally important and proposed importance-aware weighting for instruction evaluation, showing that human-perceived salience should matter when measuring compliance (Ziems et al. [2024]). More recently, MOSAIC proposed a modular benchmark for instruction compliance with up to twenty application-oriented constraints and demonstrated that compliance varies substantially by constraint type, number, and position, exposing primacy and recency effects that are invisible in coarser evaluations (Purpura et al. [2026]).

A related body of work studies instruction following under more demanding task compositions. EIFBench was explicitly designed for extremely complex instruction following, combining multiple tasks and multiple constraints to better reflect real-world prompting complexity (Zou et al. [2025]). UltraBench pushed this further in the direction of extreme fine-grained control, arguing that most prior benchmarks cover only a handful of attributes while realistic deployment often requires dense control over many text properties simultaneously (Yun et al. [2025]). Taken together, these benchmarks establish that modern evaluation must move beyond single, clean prompts with one obvious success criterion. However, they still primarily measure whether models can follow *what* is asked, not whether they remain stable when the same request is expressed in different linguistic forms (Qin et al. [2024], Ziems et al. [2024], Purpura et al. [2026], Zou et al. [2025], Yun et al. [2025]).

2.2 Constraint Following and Specialized Instruction Control

Another major line of research isolates particular instruction dimensions such as length control. Following Length Constraints in Instructions showed that even strong instruction-following models frequently violate explicit length requirements and proposed training interventions to improve controllability, highlighting that instruction compliance can fail even when the intended requirement is simple and explicit (Yuan et al. [2025]). LIFEBench then broadened the study of length instruction following across tasks, languages, and a wide range of target lengths, showing that most models handle short constraints more reliably than long ones and that they often fail to reach vendor-claimed output capacities in practice (Zhang et al. [2025a]). A different benchmark also named LIFBench focused on instruction-following performance and stability in long-context scenarios, with 11 tasks across three scenarios and a rubric-based evaluation method designed to assess both capability and stability under long inputs (Wu et al. [2025]).

These benchmarks are highly relevant because they show that instruction following is sensitive to the *type* of constraint being imposed. Length is not only a formatting issue; it reveals deeper tensions between learned generation preferences and explicit user control (Yuan et al. [2025], Zhang et al. [2025a]). Still, this literature generally varies the target requirement itself rather than the natural-language realization of that requirement. In other words, it tells us whether a model can obey a certain type of instruction, but not whether it can do so consistently under paraphrase, compression, implicitization, discourse framing, or boundary blur.

2.3 Multilingual and Cross-Lingual Instruction Following

Instruction-following evaluation has also expanded beyond English. M-IFEval extended objective instruction-following evaluation to French, Japanese, and Spanish and showed that language-specific instructions and evaluation rules matter even when the underlying task family is the same (Dussolle et al. [2025]). Marco-Bench-MIF scaled this perspective to 30 languages and emphasized localization rather than naive translation, showing sizable differences across languages and resource levels in multilingual instruction following (Zeng et al. [2025]). MaXIFE likewise introduced a large multilingual and cross-lingual instruction-following benchmark spanning 23 languages and 1,667 verifiable instruction tasks, combining rule-based and model-based evaluation to improve coverage and reliability (Liu et al. [2025]). XIFBench focused specifically on multilingual instruction following under a requirement-based framework and proposed five constraint categories across six languages, using English requirements as semantic anchors to support consistent cross-lingual evaluation (Li et al. [2025]).

This multilingual literature is directly relevant to an expanded IRBench because it demonstrates that instruction-following quality is shaped not only by task complexity, but also by localization quality, script, linguistic structure, and culturally specific expression (Dussolle et al. [2025], Zeng et al. [2025], Liu et al. [2025], Li et al. [2025]). At the same time, even the best multilingual benchmarks still concentrate on instruction transfer across languages rather than on *instruction invariance under semantically equivalent variation*. A benchmark that spans many variation families should therefore treat multilinguality as one axis of robustness, but not the only one.

2.4 Prompt Sensitivity, Paraphrase Robustness, and Benchmark Reliability

A growing literature shows that LLM behavior is sensitive to semantically preserving prompt variation. An empirical study on robustness to perturbed instructions found that character-level and word-level modifications can materially degrade model performance and also examined mitigation

strategies such as self-denoising (Agrawal et al. [2025]). Another 2026 study on prompt sensitivity explicitly quantified robustness via prompt variability, decomposing performance variance into brittleness versus task difficulty and showing that prompt perturbations can significantly affect downstream performance (Romanou et al. [2026]).

Paraphrase-specific studies reinforce the same point. Say It Another Way proposed a user-grounded paraphrase auditing framework and emphasized that automatically generated paraphrases require careful filtering for instruction adherence, semantic similarity, and realism before they can be used reliably in robustness evaluation (Chataigner et al. [2025]). Latent Adversarial Paraphrasing and RobustAlpaca showed that semantically equivalent paraphrases can induce strong worst-case degradation and motivated paraphrase-aware robustness training (Fu and Barez [2025]). Although it is outside pure text generation, LIBERO-Para offered an especially clean methodological lesson: by holding the task fixed and varying only the wording of the instruction, paraphrase robustness can be studied as a diagnostic property in its own right (Kim et al. [2026]).

This body of work provides the most direct foundation for IRBench. It supports the claim that semantic equivalence at the instruction level does not guarantee output invariance, and it also shows that paraphrase generation itself is nontrivial and must be validated carefully (Agrawal et al. [2025], Romanou et al. [2026], Chataigner et al. [2025], Fu and Barez [2025], Kim et al. [2026]). However, most of this literature still focuses on a relatively narrow perturbation family, usually paraphrasing or low-level lexical noise. A broader benchmark should extend beyond those settings into structural, pragmatic, discourse-level, and hierarchy-sensitive variation.

2.5 Instruction Hierarchy, Prompt Injection, and Boundary Robustness

Robust instruction following also depends on whether a model correctly distinguishes valid instructions from lower-priority or irrelevant text. IHEval was introduced to evaluate instruction hierarchy, i.e., whether models respect the priority of system, developer, and user instructions when they conflict (Zhang et al. [2025b]). A 2026 OpenAI paper on instruction hierarchy training data further underscored the importance of hierarchy-aware behavior by releasing IH-Challenge for improving compliance under conflicts across system, developer, user, and tool instructions (Guo et al. [2026]). At the same time, work on prompt injection robustness has shown that instruction following can break down when irrelevant or malicious text is inserted into the context, and recent work frames prompt injection as a kind of role confusion problem tied to privilege boundaries and instruction hierarchy (Jia et al. [2026], Ye et al. [2026]).

These studies are especially important for an expanded benchmark because they show that instruction robustness is not only a matter of paraphrase tolerance. It also involves the ability to maintain the correct task under boundary blur, distractor context, quoted examples, embedded narratives, and other realistic conditions in which instructions are not isolated from surrounding text (Zhang et al. [2025b], Guo et al. [2026], Jia et al. [2026], Ye et al. [2026]). Existing security-focused benchmarks capture some of this space, but usually under adversarial assumptions. A broader instruction-robustness benchmark can adapt this insight to non-adversarial, user-realistic settings such as benign distractors, narrative wrappers, and context-heavy prompts.

2.6 Gaps in Existing Literature

The literature makes clear that current evaluation has become strong along several individual axes: decomposed compliance, complex multi-constraint following, multilingual generalization, long-context stability, length control, paraphrase sensitivity, and instruction hierarchy (Qin et al. [2024], Ziems et al. [2024], Purpura et al. [2026], Zou et al. [2025], Yun et al. [2025], Dussolle et al.

[2025], Zeng et al. [2025], Liu et al. [2025], Li et al. [2025], Yuan et al. [2025], Zhang et al. [2025a], Wu et al. [2025], Agrawal et al. [2025], Fu and Barez [2025], Kim et al. [2026], Zhang et al. [2025b]). Yet these advances remain fragmented. Existing benchmarks usually optimize for one or two dimensions at a time, and very few systematically evaluate what should happen when the *same task* is expressed through a rich family of semantically equivalent instruction variants.

This leaves at least four major gaps. First, most benchmarks assume a single canonical instruction and do not test invariance under alternative realizations. Second, perturbation work typically focuses on only a narrow subset of variation such as paraphrase, synonyms, or typos, leaving pragmatic framing, instruction compression and inflation, implicitization, scope masking, benign boundary blur, and discourse wrappers underexplored. Third, most studies emphasize end-task correctness rather than the joint evaluation of correctness *and* output stability. Fourth, there is little unified evidence about which task styles are most sensitive to which instruction-variation families across the same benchmark framework (Agrawal et al. [2025], Romanou et al. [2026], Chataigner et al. [2025], Fu and Barez [2025], Kim et al. [2026], Zhang et al. [2025b]).

2.7 Implications for the Present Work

These gaps motivate a broader benchmark design than a minimal proof-of-concept. If IRBench is intended to cover the full perturbation space, it should not be limited to summarization, translation, reasoning, and extraction alone. The literature suggests that instruction robustness should be tested across multiple task styles, including classification, rewriting, structured generation, ranking, critique or verification, long-context following, and multi-turn carryover settings, because different benchmarks have shown that compliance and stability vary sharply by task type, constraint type, and context length (Purpura et al. [2026], Yun et al. [2025], Wu et al. [2025], Liu et al. [2025]). Likewise, the perturbation taxonomy should extend beyond the commonly studied families of paraphrase and noise to include discourse-level and pragmatically realistic variants that current literature has not benchmarked systematically.

Accordingly, the present project is best positioned not as another benchmark for a single instruction-following phenomenon, but as a unifying robustness benchmark for *instruction invariance under controlled variation*. Existing datasets can be reused as seed pools for canonical task instances, but the benchmark itself should be built by generating and validating a much broader set of instruction variants than prior work has covered. In that form, IRBench would connect the decompositional scoring ideas of InFoBench and TOWER, the constraint realism of EIFBench and UltraBench, the localization lessons of multilingual benchmarks, the validation lessons of paraphrase auditing, and the boundary-aware concerns of hierarchy and injection benchmarks into one unified evaluation framework (Qin et al. [2024], Ziems et al. [2024], Zou et al. [2025], Yun et al. [2025], Dussolle et al. [2025], Zeng et al. [2025], Liu et al. [2025], Li et al. [2025], Chataigner et al. [2025], Zhang et al. [2025b], Guo et al. [2026]).

3 Benchmark Architecture

IRBench is designed to evaluate whether a model remains stable when the task stays fixed but the instruction is expressed differently. To study this property in a controlled manner, the benchmark is structured around canonical task instances and systematically generated instruction variants. Each benchmark item consists of a fixed input, a canonical instruction, and an expected output or evaluation target. Multiple alternative formulations of the same instruction are then generated while preserving task semantics. A model is evaluated on the canonical form and all corresponding variants, and any deviation in output or performance is attributed to instruction variation.

Table 1: Comparison of recent instruction-following and robustness literature and how each informs the design of IRBench.

Work	Reference	Primary Focus	Strengths	Limitations for Our Setting	Takeaway for IRBench
InFoBench	Qin et al. [2024]	Decomposed instruction evaluation	Introduces DRFR for fine-grained evaluation; decomposes instructions into atomic requirements; improves interpretability	Assumes canonical instructions; does not evaluate robustness under phrasing variation	Use decomposition idea for validating semantic equivalence and fine-grained scoring
TOWER	Ziems et al. [2024]	Importance-weighted instruction evaluation	Models importance hierarchy within instructions; aligns evaluation with human perception	Focuses on weighting rather than robustness to linguistic variation	Use importance-aware weighting for evaluating variant correctness
MOSAIC	Purpura et al. [2026]	Modular constraint-based evaluation	Evaluates up to many constraints; reveals primacy/recency effects in instruction following	Synthetic constraints; does not vary instruction phrasing systematically	Use constraint interaction insights for perturbation difficulty analysis
EIFBench	Zou et al. [2025]	Multi-instruction multi-constraint tasks	Realistic complex instruction scenarios; shows models fail joint constraints	Uses clean instructions; no robustness to variation	Use complex task setups with perturbed instruction variants
UltraBench	Yun et al. [2025]	Fine-grained generation control	Expands evaluation across many controllable attributes	Focuses on constraint satisfaction, not invariance	Use fine-grained control tasks as canonical seeds
LIFEBench (Length)	Zhang et al. [2025a]	Length-constrained instruction following	Studies difficulty across output lengths and tasks	Narrow to length constraints; no linguistic variation	Extend to robustness under varied phrasing of constraints
LIFBench (Long Context)	Wu et al. [2025]	Long-context instruction following	Evaluates stability under long inputs and complex contexts	Does not isolate instruction phrasing effects	Add long-context robustness as an analysis dimension
M-IFEval	Dussolle et al. [2025]	Multilingual instruction following	Extends evaluation across multiple languages	Focuses on language transfer, not phrasing variation	Extend IRBench to multilingual perturbation variants
Marco-Bench-MIF	Zeng et al. [2025]	Large-scale multilingual evaluation	Covers many languages; highlights localization issues	Does not test semantic invariance under instruction variation	Use cross-lingual variants as an extension of perturbations
XIFBench	Li et al. [2025]	Multilingual constraint-based evaluation	Uses requirement-based framework across languages	No robustness analysis under instruction reformulation	Combine multilingual + robustness axes in IRBench
Perturbed Instructions	Agrawal et al. [2025]	Robustness to noise and perturbations	Shows small perturbations degrade performance significantly	Limited to low-level perturbations (typos, edits)	Extend to structural, discourse, and pragmatic perturbations
LAP	Fu and Barez [2025]	Adversarial paraphrase robustness	Demonstrates worst-case degradation under paraphrasing	Focused mainly on paraphrase	Extend to multi-family perturbation space beyond paraphrase
Say It Another Way	Chataigner et al. [2025]	Paraphrase validation and filtering	Highlights need for semantic equivalence validation	Focuses on paraphrase only	Use validation pipeline for all perturbation families
LIBERO-Para	Kim et al. [2026]	Paraphrase robustness under controlled tasks	Isolates instruction variation as only variable	Domain-specific (robotics / VLA)	Adopt core idea: fix task, vary instruction
IHEval	Zhang et al. [2025b]	Instruction hierarchy evaluation	Tests priority handling across instruction sources	Not designed for linguistic variation robustness	Add hierarchy-aware perturbation family
IH-Challenge	Guo et al. [2026]	Instruction hierarchy robustness training	Focuses on conflict resolution across instruction levels	Limited to hierarchical conflicts	Extend to non-adversarial boundary blur variants
Prompt Injection / Role Confusion	Ye et al. [2026]	Instruction boundary and injection robustness	Shows models fail under conflicting or injected instructions	Focuses on adversarial settings	Introduce benign distractor and boundary perturbations

At a high level, the benchmark consists of four components. The first component is the *canonical instance layer*, which defines task instances using multiple canonical instruction templates. The second component is the *variation layer*, which generates semantically equivalent instruction variants based on a structured perturbation taxonomy. The third component is the *execution layer*, where models are evaluated under controlled conditions across canonical and perturbed instructions. The fourth component is the *evaluation layer*, which measures both task correctness and behavioral stability across instruction variations.

Formally, each benchmark instance is represented as:

$$\mathcal{B}_j = (x_j, \mathcal{I}_j^{(0)}, \mathcal{V}(\mathcal{I}_j^{(0)}), y_j^*, m_j), \tag{1}$$

where x_j is the input, $\mathcal{I}_j^{(0)}$ is a set of canonical instruction templates, $\mathcal{V}(\mathcal{I}_j^{(0)})$ denotes the set of instruction variants derived from these templates, y_j^* is the reference output or evaluation target, and m_j is metadata including task type, perturbation category, and generation source.

The benchmark is constructed such that only the instruction varies. The input remains fixed, the task definition remains fixed, and the evaluation procedure remains fixed. This design ensures that any observed variation in model outputs can be attributed specifically to differences in instruction phrasing.

3.1 Canonical Instance Layer

The benchmark begins with canonical instance construction. Each instance consists of an input example paired with a set of canonical instruction templates and a corresponding output or evaluation target. Unlike traditional setups that rely on a single instruction, IRBench defines multiple canonical templates per task to reduce bias toward any particular phrasing.

Each canonical instruction is designed to express the same task while varying in linguistic form, including directive, descriptive, interrogative, and reformulated styles. This multi-template design ensures that robustness is evaluated relative to a distribution of valid instructions rather than a single reference form.

To ensure consistency, canonical instructions are constructed to satisfy three criteria: (i) the task is explicitly defined, (ii) all necessary requirements are specified, and (iii) the expected output format is clear and scorable.

3.2 Instruction Variation Layer

For each canonical instruction template, IRBench generates multiple instruction variants based on a predefined perturbation taxonomy. These perturbations include surface-level changes, structural transformations, semantic-preserving paraphrases, pragmatic framing variations, benign distractor additions, compression and expansion, implicit formulations, scope masking, boundary blurring, register shifts, and cognitive load variations.

Each variant is associated with a perturbation category and generation method, allowing the benchmark to analyze robustness across distinct dimensions of instruction variation. Importantly, these transformations are designed to preserve task semantics while introducing variation in linguistic form, discourse structure, or contextual framing.

3.3 Execution Layer

In the execution phase, each model is evaluated on the same input under all canonical instruction templates and their corresponding variants. Decoding settings are controlled to minimize stochastic

variation, ensuring that observed differences in output can be attributed to instruction phrasing rather than randomness in generation.

All outputs are stored along with the instruction used, the perturbation category, and model configuration details. This produces a structured representation of model behavior under controlled instruction variation.

3.4 Evaluation Layer

The evaluation layer compares model outputs across canonical and perturbed instructions. Evaluation is performed along two dimensions. The first dimension measures task correctness under each instruction variant. The second dimension measures consistency relative to outputs produced under canonical instructions.

This dual evaluation framework allows IRBench to distinguish between models that maintain task performance but vary in output behavior and those that degrade in correctness under instruction variation. The architecture is therefore designed to support both accuracy-based and stability-based analysis of instruction robustness.

4 Instruction-Variation Taxonomy

A central contribution of IRBench is a broad and structured taxonomy of instruction variation. Existing robustness work often focuses on a narrow class of perturbations such as paraphrases, typographical noise, or adversarial injections. IRBench instead treats instruction variation as a multi-dimensional phenomenon. The taxonomy is designed to capture not only lexical or syntactic changes, but also discourse framing, implicitness, boundary clarity, and user-style diversity. Each category is defined so that the task remains unchanged even though the surface form of the instruction is altered.

4.1 Surface-Level Variations

Surface-level variations modify the visible form of the instruction without changing its syntactic structure or core wording. These perturbations are intended to model the kinds of imperfections that appear in natural typing, copied text, mobile input, or casual user behavior.

This category includes typographical errors, missing or repeated characters, punctuation changes, spacing irregularities, capitalization shifts, and minor grammatical slips. For example, a canonical instruction such as “Summarize the following article in one paragraph” may become “Summarize teh following article in one paragraph” or “summarize the following article in one paragraph.” These changes do not alter task semantics, but they test whether a model is brittle to visually noisy input.

This category is useful because it establishes a low-level robustness baseline. A model that fails under small orthographic distortions is unlikely to be robust under more sophisticated variations. At the same time, this category should not dominate the benchmark, because it captures only one limited form of instruction variation.

4.2 Structural Variations

Structural variations preserve the meaning of the instruction while changing its syntactic arrangement. The purpose of this category is to test whether a model is sensitive to sentence structure rather than only to task semantics.

This category includes clause reordering, active-passive alternation, nominalization, sentence splitting, sentence merging, and modifier repositioning. For example, "Extract the names of all organizations mentioned in the text" may be rewritten as "From the text, extract all mentioned organization names" or "The names of all organizations mentioned in the text should be extracted." The intended task remains the same, but the syntactic packaging changes.

Structural variation is important because many instruction-following models encounter common phrasings repeatedly during training and may over-rely on familiar syntactic templates. A robustness benchmark should therefore test whether the model recognizes task equivalence even when syntax shifts substantially.

4.3 Semantic-Preserving Paraphrases

Semantic-preserving paraphrases alter lexical realization more substantially while maintaining the same underlying task. This category is broader than simple synonym replacement and aims to capture realistic alternative phrasings that a user might naturally produce.

Examples include replacing "summarize" with "give a brief overview," replacing "translate" with "render in German," or rewriting an extraction request with a more conversational expression. Multi-hop paraphrase chains can also be used, where an instruction is paraphrased several times in succession to produce a more distant but still equivalent variant.

This category is one of the most important because it most directly tests the central benchmark question: whether semantically equivalent instructions lead to stable model behavior. It also provides a bridge to prior robustness literature while allowing IRBench to extend beyond paraphrase-only evaluation.

4.4 Pragmatic Framing Variations

Pragmatic framing variations preserve the requested task but change the discourse stance of the instruction. This category is intended to capture the fact that users do not all issue instructions in the same communicative mode. Some ask politely, some give direct commands, and some phrase requests as suggestions or collaborative prompts.

Examples include converting a direct instruction into a polite request, changing a request into a command, or changing a neutral instruction into a more formal or conversational framing. A canonical form such as "Classify the sentiment of the review as positive or negative" might become "Could you tell me whether this review is positive or negative?" or "Please determine the sentiment label for the following review."

This category is important because it tests whether a model is driven by the task itself or by pragmatic signals associated with politeness, directness, or social framing. It also reflects real user diversity more accurately than purely syntactic benchmarks.

4.5 Instruction Compression Variations

Instruction compression reduces the canonical instruction to a shorter and more telegraphic form while preserving the same meaning. The purpose of this category is to test whether a model can recover task intent when explicit scaffolding is reduced.

Examples include dropping function words, shortening expressions, or converting full sentences into fragments. "Summarize the following article in one paragraph" may become "One-paragraph summary of the article below" or simply "Article summary, one paragraph." These compressed forms are common in note-taking, search-box style prompting, or quick user interactions.

Compressed instructions are especially useful because they reveal whether a model depends on full grammatical completeness. A robust model should still recognize the task even when the instruction is reduced to its essential semantic content.

4.6 Instruction Expansion Variations

Instruction expansion does the opposite of compression. It rewrites the same instruction in a more verbose and elaborated manner while preserving the task. The goal is to test whether a model remains stable when the same request is embedded in additional but semantically redundant wording.

For example, "Translate the following sentence into French" may become "Please read the sentence shown below and provide its translation in French, making sure the output is only the translated sentence." These expanded forms can include explanatory setup, redundant restatement, or additional politeness phrases, as long as they do not change the underlying task.

This category matters because many real prompts are overexplained, especially in educational, workplace, or high-stakes settings. A robust model should not change behavior merely because the same request is expressed at greater length.

4.7 Implicit Instruction Variations

Implicit instruction variations reduce the explicitness of the directive while keeping the intended task recoverable from context. This category captures a form of natural instruction use that is common in conversational settings, where users often imply rather than fully specify the action they want.

A canonical instruction such as "Extract all dates mentioned in the passage" may become "Dates mentioned in the passage" or "All dates from the passage below." Similarly, "Summarize the text in two sentences" may become "Two-sentence summary." These are not malformed prompts; they are concise, human-like elliptical instructions.

This category is particularly valuable because it tests whether a model can infer task intent when imperative structure is weakened. It also differentiates robustness to incomplete wording from robustness to simple noise.

4.8 Scope-Masking Variations

Scope-masking variations preserve the target task but make the scope of the task less explicitly signaled. The purpose is to test whether a model can still identify what should be operated on, extracted, classified, or transformed when the instruction is less explicit about scope boundaries.

For instance, "Identify the organizations mentioned in the following article" could become "Which organizations appear here?" or "Organizations mentioned below." The task remains the same, but the explicit mention of extraction boundaries or item type is softened or partially hidden.

This category is useful because many real instructions are underspecified in exactly this way. Users often expect the model to infer the relevant target set from context rather than stating all scope cues explicitly.

4.9 Instruction-Context Boundary Blur

Instruction-context boundary blur embeds the instruction within surrounding text so that the boundary between directive and context is less clean. This category is designed to reflect real-

istic prompts where a user includes setup, background, or examples before or around the actual instruction.

For example, a user may write a sentence explaining why they need help before stating the task, or may quote an example and then ask for a similar transformation. The key constraint is that the extra context must be benign and must not change the underlying task.

This category is important because most real prompts are not laboratory-clean. A benchmark that only evaluates isolated one-line instructions may overestimate model robustness by ignoring the difficulty of separating instruction from surrounding discourse.

4.10 Benign Distractor Variations

Benign distractor variations add irrelevant but non-adversarial information to the prompt. Unlike prompt injection benchmarks, the goal here is not to attack the model, but to simulate ordinary conversational clutter.

Examples include prefacing the instruction with remarks such as "I need this for class," "I'm in a hurry," or "This might be obvious, but please help." These additions should not change the task, but they may affect how the model formats or prioritizes its answer.

This category is useful because it captures a realistic failure mode: a model that is overly sensitive to contextual clutter may drift in tone, output format, or content even when the task itself has not changed.

4.11 Register and Style Transfer Variations

Register and style transfer variations rewrite the instruction in different sociolinguistic styles while preserving intent. This category captures user diversity across age, background, profession, and linguistic fluency.

Examples include rewriting instructions in a professional tone, an informal conversational tone, a student-like tone, or a non-native but understandable English style. For instance, "Please classify this review as positive or negative" might become "Can you check if this review is good or bad?" or "Determine the sentiment polarity of the following review."

This category is especially valuable because robustness should not be defined only with respect to one standardized variety of English. A model intended for broad real-world use should remain stable across stylistic and register differences.

4.12 Cognitive Load Variations

Cognitive load variations increase the apparent complexity of the instruction without changing the task itself. This is done by adding semantically redundant qualifiers, subordinate clauses, or explanatory phrasing that makes the instruction more effortful to parse.

For example, a short summarization instruction may be rewritten as a longer sentence with nested clauses and redundant guidance, yet still request exactly the same output. The purpose of this category is to test whether the model becomes less stable when the same task is presented in a cognitively heavier form.

This category is novel and useful because it distinguishes between semantic difficulty and presentation difficulty. A model may know the task perfectly well, but still become inconsistent when the directive is wrapped in syntactically or discourse-level complexity.

4.13 Why a Broad Taxonomy is Necessary

This taxonomy is deliberately broader than existing narrow perturbation sets because instruction sensitivity is not a single phenomenon. A model may handle typos well but fail under implicit phrasing, remain stable under paraphrase but drift under verbose discourse framing, or succeed under direct commands but fail under boundary blur. Treating all variations as one undifferentiated pool would hide these patterns. By separating variation families, IRBench can identify not only whether a model is instruction-sensitive, but also *how* and *where* that sensitivity appears.

5 Dataset Construction

We construct IRBench by applying controlled instruction variations to existing task datasets while keeping the underlying task, input, and evaluation criteria fixed. This design isolates instruction phrasing as the only source of variation, allowing us to attribute changes in model behavior directly to linguistic differences in instructions.

Each benchmark instance is defined by a canonical tuple (x, I_0, y^*), where x is the input, I_0 is the canonical instruction, and y^* is the reference output or evaluation target. From this canonical form, we generate a set of instruction variants $\mathcal{V}(I_0)$ that preserve task semantics while differing in linguistic realization.

5.1 Canonical Instance Construction

We instantiate IRBench using publicly available datasets spanning multiple task families, including summarization, translation, reasoning, extraction, and classification. For each task, we define a canonical instruction that explicitly specifies the task requirements and expected output format.

Canonical instructions are manually designed to ensure clarity, completeness, and neutrality. They serve as the semantic reference against which all instruction variants are evaluated. Importantly, canonical instructions are constructed to avoid stylistic bias toward any particular phrasing pattern, enabling meaningful variation across perturbation categories.

5.2 Instruction Variant Generation

Instruction variants are generated through a combination of rule-based transformations and model-assisted rewriting. Rule-based methods are used for systematic perturbations such as surface-level modifications, structural transformations, and controlled compression or expansion. Model-assisted methods are used for generating paraphrases, stylistic variations, and discourse-level transformations.

Each variant is assigned a perturbation label corresponding to the taxonomy defined in Section X. This labeling enables structured analysis of model robustness across different categories of instruction variation.

5.3 Semantic Validation

To ensure that instruction variants preserve the intended task, we introduce a validation stage that filters out variants that alter task semantics. This stage combines automated filtering with manual verification.

Automated filtering removes malformed or redundant variants and enforces basic similarity constraints relative to the canonical instruction. Manual validation ensures that each variant pre-

serves the original task intent, does not introduce new constraints, and does not remove required conditions.

Only validated variants are included in the final benchmark.

5.4 Dataset Instantiation

Following validation, each benchmark instance is represented as:

$$(x, I_0, \mathcal{V}(I_0), y^*, m), \tag{2}$$

where m contains metadata including task type, perturbation category, and generation method.

We construct the dataset using a balanced sampling strategy across task types and perturbation categories to ensure sufficient coverage for comparative analysis.

6 Experimental Setup

We evaluate IRBench on a set of open-source language models under a controlled instruction-variation setting. For each benchmark instance, a model is evaluated on four canonical instruction templates, denoted by C_1, C_2, C_3, and C_4, and on the perturbation variants derived from each canonical template. This setup allows us to measure not only task performance, but also the degree to which model behavior remains stable across multiple valid ways of expressing the same instruction.

6.1 Models

We consider a diverse set of open-source models spanning multiple model families and parameter scales, including LLaMA 2 **?**, Mistral **?**, Qwen **?**, and Gemma **?**. Using models from different families is important because robustness under instruction variation may depend not only on scale, but also on training mixture, instruction tuning strategy, and decoding behavior.

6.2 Evaluation Protocol

For each dataset instance, we first evaluate the model under the four canonical instruction templates associated with that task. We then evaluate the same instance under all perturbation variants generated from those canonical templates. The input, expected task, and decoding configuration are kept fixed throughout. In this way, any change in model behavior can be attributed to instruction variation rather than to changes in content or generation settings.

All models are evaluated under the same decoding regime. We use deterministic decoding whenever possible so that variability induced by sampling does not obscure the effect of instruction phrasing. Outputs are stored together with the task label, canonical template identifier, perturbation family, and model configuration.

6.3 Metrics

We measure model behavior using two complementary groups of metrics: *task-performance metrics* and *robustness metrics*. The task-performance metrics quantify whether the model solves the task correctly. The robustness metrics quantify whether the model remains stable when the same task is expressed through different instruction forms.

Table 2: Dataset-wise mapping of multiple canonical instruction templates and their coverage across perturbation categories in IRBench. Each dataset is associated with diverse canonical formulations to reduce dependence on a single phrasing prior to perturbation.

Task	**Dataset**	**Canonical Instruction Templates**	**Perturbation Categories**											
			Surf	**Struc**	**Para**	**Prag**	**Comp**	**Exp**	**Impl**	**Scope**	**Blur**	**Distr**	**Reg**	**Cog**
Summarization	XSum (Narayan et al. [2018]) CNN/DailyMail (See et al. [2017]) GovReport (Huang et al. [2021])	C1: Summarize the article in one sentence C2: Provide a one-sentence summary capturing the main idea C3: Write a concise overview of the text in a single sentence C4: What is the main point of the article in one sentence?	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Translation	WMT14 (Bojar et al. [2014]) WMT21 (Akhbardeh et al. [2021]) FLORES-200 (Costa-Jussà et al. [2022])	C1: Translate the sentence into German C2: Convert the following sentence into German C3: Provide the German equivalent of the given sentence C4: How would this sentence be expressed in German?	✓	✓	✓	✓	✓	✓			✓	✓	✓	
Reasoning	GSM8K (Cobbe et al. [2021]) MATH (Hendrycks et al. [2021]) BBH (Suzgun et al. [2023])	C1: Solve the math problem and provide the answer C2: Determine the correct solution to the problem C3: Compute the answer for the given math question C4: What is the final answer to this problem?	✓	✓	✓	✓	✓	✓	✓		✓	✓	✓	✓
Extraction	SQuAD (Rajpurkar et al. [2016]) SQuAD 2.0 (Rajpurkar et al. [2018]) Natural Questions (Kwiatkowski et al. [2019])	C1: Answer the question based on the passage C2: Based on the passage, answer the question C3: Extract the answer from the given context C4: What is the answer according to the passage?	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	
Classification	SST-2 (GLUE) (Wang et al. [2018]) AG News (Zhang et al. [2015])	C1: Classify the sentiment as positive or negative C2: Determine whether the sentiment is positive or negative C3:Identify the sentiment polarity of the sentence C4: Is the sentiment expressed in the text positive or negative?	✓	✓	✓	✓	✓	✓	✓			✓	✓	

6.3.1 Task-performance metrics

7 Analysis

7.1 Sensitivity to Instruction Variation

7.2 Model Size vs Robustness

7.3 Failure Cases

8 Discussion

8.1 Implications for LLM Deployment

8.2 Limitations

8.3 Future Directions

9 Conclusion

References

Aryan Agrawal, Lisa Alazraki, Shahin Honarvar, and Marek Rei. Enhancing llm robustness to perturbed instructions: An empirical study. *arXiv preprint arXiv:2504.02733*, 2025.

Farhad Akhbardeh, Arkady Arkhangorodsky, Magdalena Biesialska, Ondřej Bojar, Rajen Chatterjee, Vishrav Chaudhary, Marta R Costa-jussa, Cristina España-Bonet, Angela Fan, Christian Federmann, et al. Findings of the 2021 conference on machine translation (wmt21). In *Proceedings of the sixth conference on machine translation*, pages 1–88, 2021.

Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, et al. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the ninth workshop on statistical machine translation*, pages 12–58, 2014.

Cléa Chataigner, Rebecca Ma, Prakhar Ganesh, Afaf Taïk, Elliot Creager, and Golnoosh Farnadi. Say it another way: A framework for user-grounded paraphrasing. *arXiv e-prints*, pages arXiv–2505, 2025.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems, 2021. *URL https://arxiv. org/abs/2110.14168*, 9, 2021.

Marta R Costa-Jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*, 2022.

Antoine Dussolle, A Cardeña, Shota Sato, and Peter Devine. M-ifeval: Multilingual instruction-following evaluation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6161–6176, 2025.

Tingchen Fu and Fazl Barez. Same question, different words: A latent adversarial framework for prompt robustness. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 31293–31307, 2025.

Chuan Guo, Juan Felipe Ceron Uribe, Sicheng Zhu, Christopher A Choquette-Choo, Steph Lin, Nikhil Kandpal, Milad Nasr, Sam Toyer, Miles Wang, Yaodong Yu, et al. Ih-challenge: A training dataset to improve instruction hierarchy on frontier llms. *arXiv preprint arXiv:2603.10521*, 2026.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. Efficient attentions for long document summarization. In *Proceedings of the 2021 conference of the north American chapter of the association for computational linguistics: Human language technologies*, pages 1419–1436, 2021.

Yuqi Jia, Ruiqi Wang, Xilong Wang, Chong Xiang, and Neil Gong. Alignsentinel: Alignment-aware detection of prompt injection attacks. *arXiv preprint arXiv:2602.13597*, 2026.

Chanyoung Kim, Minwoo Kim, Minseok Kang, Hyunwoo Kim, and Dahuin Jung. Libero-para: A diagnostic benchmark and metrics for paraphrase robustness in vla models. *arXiv preprint arXiv:2603.28301*, 2026.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Zhenyu Li, Kehai Chen, Yunfei Long, Xuefeng Bai, Yaoyin Zhang, Xuchen Wei, Juntao Li, and Min Zhang. Xifbench: Evaluating large language models on multilingual instruction following. *arXiv preprint arXiv:2503.07539*, 2025.

Yile Liu, Ziwei Ma, Xiu Jiang, Jinglu Hu, ChangJing ChangJing, and Liang Li. Maxife: Multilingual and cross-lingual instruction following evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14252–14332, 2025.

Shashi Narayan, Shay B Cohen, and Mirella Lapata. Don't give me the details, just the summary. *Topic-aware convolutional neural networks for extreme summarization. ArXiv abs/1808.08745*, 2018.

Alberto Purpura, Li Wang, Sahil Badyal, Gene Beaufrand, and Adam Faulkner. Deconstructing instruction-following: A new benchmark for granular evaluation of large language model instruction compliance abilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1334–1349, 2026.

Yiwei Qin, Kaiqiang Song, Yebowen Hu, Wenlin Yao, Sangwoo Cho, Xiaoyang Wang, Xuansheng Wu, Fei Liu, Pengfei Liu, and Dong Yu. Infobench: Evaluating instruction following ability in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13025–13048, 2024.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 2383–2392, 2016.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, 2018.

Angelika Romanou, Mark Ibrahim, Candace Ross, Chantal Shaib, Kerem Okta, Sam Bell, Elia Ovalle, Jesse Dodge, Antoine Bosselut, Koustuv Sinha, et al. Brittlebench: Quantifying llm robustness via prompt sensitivity. *arXiv preprint arXiv:2603.13285*, 2026.

Abigail See, Peter J Liu, and Christopher D Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1073–1083, 2017.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*, pages 353–355, 2018.

Xiaodong Wu, Minhao Wang, Yichen Liu, Xiaoming Shi, He Yan, Lu Xiangju, Junmin Zhu, and Wei Zhang. Lifbench: Evaluating the instruction following performance and stability of large language models in long-context scenarios. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16445–16468, 2025.

Charles Ye, Jasmine Cui, and Dylan Hadfield-Menell. Prompt injection as role confusion. *arXiv preprint arXiv:2603.12277*, 2026.

Weizhe Yuan, Ilia Kulikov, Ping Yu, Kyunghyun Cho, Sainbayar Sukhbaatar, Jason E Weston, and Jing Xu. Following length constraints in instructions. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24243–24254, 2025.

Longfei Yun, Letian Peng, and Jingbo Shang. Ultrabench: Benchmarking llms under extreme fine-grained text generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15438–15453, 2025.

Bo Zeng, Chenyang Lyu, Sinuo Liu, Mingyan Zeng, Minghao Wu, Xuanfan Ni, Tianqi Shi, Yu Zhao, Yefeng Liu, Chenyu Zhu, et al. Marco-bench-mif: On multilingual instruction-following capability of large language. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24058–24072, 2025.

Wei Zhang, Zhenhong Zhou, Kun Wang, Junfeng Fang, Yuanhe Zhang, Rui Wang, Ge Zhang, Xavier Li, Li Sun, Lingjuan Lyu, et al. Lifebench: Evaluating length instruction following in large language models. *arXiv preprint arXiv:2505.16234*, 2025a.

Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28, 2015.

Zhihan Zhang, Shiyang Li, Zixuan Zhang, Xin Liu, Haoming Jiang, Xianfeng Tang, Yifan Gao, Zheng Li, Haodong Wang, Zhaoxuan Tan, et al. Iheval: Evaluating language models on following the instruction hierarchy. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8374–8398, 2025b.

Noah Ziems, Zhihan Zhang, and Meng Jiang. Tower: Tree organized weighting for evaluating complex instructions. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 13803–13810, 2024.

Tao Zou, Xinghua Zhang, Haiyang Yu, Minzheng Wang, Fei Huang, and Yongbin Li. Eifbench: Extremely complex instruction following benchmark for large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20941–20964, 2025.